Hybrid TER Partition Backend

v2.0-CODEX - Radix Histogramming + Portable IPS2Ra-Derived Blockwise In-Place Redistribution

Subset Sum TER project

24 August 2026

Contents

1 Normative purpose and decision freeze **2**
1.1 Frozen backend priority . . . 2

2 What TER actually needs **3**

3 Speed and memory math before implementation **3**
3.1 CurrentScatter auxiliary memory . . . 4

4 Repository grounding **5**
4.1 TER repository facts that constrain this design . . . 5
4.2 Donor `ssp_oracle.cpp` facts . . . 5

5 Exact TER movement record **5**

6 Exact one-chunk semantics **6**

7 Public API - frozen **6**

8 Input validation and statuses **8**

9 CurrentScatter backend **8**
9.1 Serial path . . . 8
9.2 Parallel path . . . 8
9.3 CurrentScatter physical widths . . . 9

10 BlockInPlace backend - source boundary **9**
10.1 IPS2Ra source pieces that are in scope . . . 9
10.2 IPS2Ra pieces explicitly out of scope . . . 9

11 BlockInPlace v1 physical design **9**

12 BlockInPlace algorithm - exact phase contract **10**
12.1 Phase BI-1: local classification and buffered flushing . . . 10
12.2 Phase BI-2: global bucket boundaries . . . 10
12.3 Phase BI-3: bucket block ownership . . . 10
12.4 Phase BI-4: full-block permutation . . . 11
12.5 Phase BI-5: partial-boundary cleanup . . . 11

13 Worker pool and portability **11**

14 PartitionWorkspace internals **11**

15 Memory admission **12**
15.1 CurrentScatter estimate . . . 12

15.2 BlockInPlace estimate . . . 12

16 Telemetry - common and backend-specific **12**

17 Correctness protocol **13**
17.1 Differential oracle . . . 13

18 Benchmark protocol **14**
18.1 Dataset matrix . . . 14
18.2 Paired timing . . . 14
18.3 Promotion gates . . . 14

19 Adaptive backend - deferred contract **14**

20 Region2Path - deferred comparator contract **15**

21 Licensing and source-provenance rules **15**

22 File layout - frozen recommendation **15**

23 Build integration **16**

24 Implementation order and stop gates **16**
24.1 H0 - safety audit . . . 16
24.2 H1 - API + status + tests . . . 16
24.3 H2 - CurrentScatter serial . . . 16
24.4 H3 - CurrentScatter parallel . . . 16
24.5 H4 - isolated benchmark . . . 16
24.6 H5 - BlockInPlace serial reference . . . 16
24.7 H6 - BlockInPlace parallel . . . 16
24.8 H7 - paired benchmark and decision . . . 16
24.9 H8 - root Stage C integration . . . 17

25 Absolute no-go list **17**

26 Final P2 acceptance checklist **17**

1 Normative purpose and decision freeze

This document is the **Stage C / P2 implementation specification** for the TER L1 -> L0 root. It replaces the earlier research-oriented hybrid report as the coding authority. The earlier report remains useful evidence, but this edition freezes the engineering decisions required to prevent Codex from interpreting several experimental alternatives as equally urgent production work.

Master architecture: `TER_L1_to_L0_Radix_Refined_Root_Architecture_v2.1_CODEX.pdf`.

Executable task sequence: `implementation_plan.md`.

Committed TER baseline: `Namanpm/subset-sum-ter-fft` commit `3eac235f871c2e4cf07a73152644e6113eaf39e1`.

Local donor: the uncommitted `ssp_oracle.cpp` copied into the TER worktree. This file is the radix-code donor and MUST be inspected locally. Codex MUST NOT substitute a GitHub or File Library copy when implementing if the local file differs.

Read-only external reference: `C:\Users\naman\Desktop\ips2ra`, upstream `ips4o/ips2ra`, reference HEAD `ef3407632348f58a6ab75832a749cde99f3b9ad8` as of this specification.

1.1 Frozen backend priority

The implementation order is:

```
1. CurrentScatter   REQUIRED FIRST
2. BlockInPlace     PRIMARY IN-PLACE CANDIDATE
3. Region2Path      DEFERRED COMPARATOR
4. Adaptive         DEFERRED UNTIL MEASURED DATA
```

This is a deliberate change from the earlier report, which placed Region2Path before BlockInPlace in its milestone list. The later engineering evidence and current Windows integration constraints make IPS2Ra-style block permutation the higher-value implementation target.

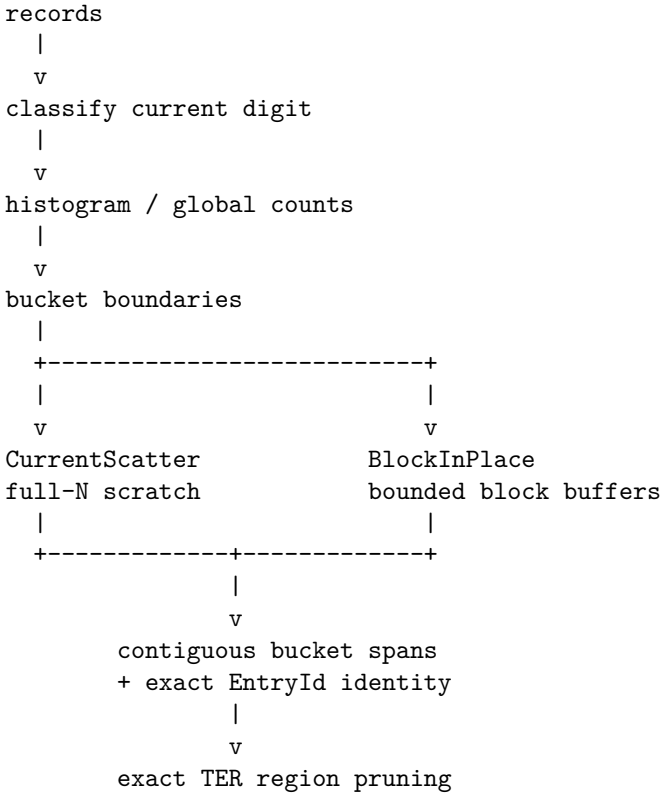
Codex MUST NOT implement Region2Path before the CurrentScatter + BlockInPlace bake-off is correct and measured unless this document is revised.

2 What TER actually needs

TER does not need a general-purpose total sort. It needs one operation:

> Given compact records carrying an exact projected quotient key and an EntryId, partition the array by one requested physical digit, return contiguous bucket spans, preserve every EntryId exactly once, and stop.

The abstraction is **partition**, not sort.



3 Speed and memory math before implementation

No measured TER speedup exists for the new backend yet. The implementation MUST begin from the algebra and benchmark rather than from a claimed factor.

Let:

- $H = T_{histogram} + T_{prefix}$;
- $S = T_{scatter}$ for CurrentScatter;
- $R = T_{local\ classification} + T_{block\ permutation} + T_{cleanup}$ for BlockInPlace.

Then:

$$T_{current} = H + S,$$
$$T_{block} = H + R,$$
$$Speedup = \frac{H + S}{H + R}.$$

Break-even is $R < S$.

If $H/S = 0.5$:

R/S	Modelled current/block speedup
0.50	1.50x
0.70	1.25x
0.80	1.15x
0.90	1.07x
1.00	1.00x
1.20	0.88x
1.50	0.75x

These values are **not benchmark results**.

3.1 CurrentScatter auxiliary memory

For N records of size $s_e = 8$ bytes, t workers, $B = 2^b$, and 8-byte `size_t`:

$$M_{worker} = 2tB \cdot 8,$$
$$M_{global} = B \cdot 8,$$
$$M_{scratch} = Ns_e.$$

At 16 workers:

bits	buckets	worker histogram+offset	global	dense metadata total
8	256	0.0625 MiB	0.0020 MiB	0.0645 MiB
12	4,096	1.00 MiB	0.0313 MiB	1.0313 MiB
16	65,536	16.00 MiB	0.50 MiB	16.50 MiB
24	16,777,216	4,096 MiB	128 MiB	4,224 MiB

Full record scratch for 8-byte records:

N	scratch
20,000	0.153 MiB
32,496	0.248 MiB
64,000	0.488 MiB
0.5M	3.815 MiB
1M	7.629 MiB
2M	15.259 MiB
5M	38.147 MiB

This is why the in-place backend is primarily a **memory-traffic/scratch candidate**, not an automatic speed win.

4 Repository grounding

4.1 TER repository facts that constrain this design

The current TER repository is C++20, uses strict warnings, centralizes numeric runtime policy in `include/ter/limitations.hpp`, and has no IPS2Ra/TBB dependency in `CMakeLists.txt`. The current root already exposes:

- `FullTerRootCapture` with genuine L1 lists and little-endian residue words;
- `RootOutputPolicy` with `FirstWitness`, `CountOnly`, and `FullCanonicalOutput`;
- root prefilter/tile config fields;
- stage telemetry including pair probes/records, quotient width, prefilter candidates, exact rejects, output memory, and early stop;
- root report/count APIs for isolated root replay.

Therefore P2 can be implemented and benchmarked without inventing a new capture mechanism.

4.2 Donor `ssp_oracle.cpp` facts

The local donor contains the mature radix pieces we want:

```
struct alignas(64) RadixWorkerState {
  std::vector<size_t> histogram;
  std::vector<size_t> offsets;
};
```

It also contains a persistent `std::thread` worker pool and a generic unsigned radix sorter. The parallel path already records:

```
sort_threads_requested
sort_threads_used
parallel_radix_passes
radix_histogram_bytes
radix_offsets_bytes
radix_parallel_count_ns
radix_prefix_ns
radix_parallel_scatter_ns
```

The donor scatter conceptually performs:

```
value = values[index];
bucket = (value >> shift) & mask;
scratch[offset++] = value;
```

The required TER adaptation is:

```
record = entries[index];
bucket = (record.low_k >> shift) & mask;
scratch[offset++] = record;
```

The key change is identity-preserving record movement, not a new mathematical sorter.

5 Exact TER movement record

The canonical aliases already exist in `include/ter/types.hpp`:

```
using EntryId = std::uint32_t;
using QuotientResidue = std::uint32_t;
```

The partition movement record is frozen as:

```
struct TerPartitionEntry {
  QuotientResidue low_k;
  EntryId entry_id;
};
```

```cpp
static_assert(sizeof(TerPartitionEntry) == 8);
static_assert(std::is_trivially_copyable_v<TerPartitionEntry>);
```

`low_k` is the only classification key. `entry_id` is payload and MUST travel unchanged with its key.

Do not move:

- arbitrary-width residue words;
- packed ternary digits;
- provenance objects;
- full parent entries;
- a duplicate full-width quotient.

6 Exact one-chunk semantics

For `entries[0..N)`, `shift=s`, `bits=b`, $B = 2^b$:

$$digit(e) = ((e.low_k \gg s) \ \& \ (B-1)).$$

After successful partition:

- every digit occupies one contiguous span;
- `offsets[j]..offsets[j+1]` contains exactly digit `j`;
- no record is dropped or duplicated;
- no EntryId is changed;
- records inside one bucket need not be stable or sorted;
- the function stops after this digit.

This is stricter and simpler than a full LSD/MSD sort.

7 Public API - frozen

Create `include/ter/root_partition.hpp` with the following public contract:

```cpp
#pragma once

#include "ter/types.hpp"

#include <cstdint>
#include <span>
#include <type_traits>
#include <vector>

namespace ter {

struct TerPartitionEntry {
  QuotientResidue low_k = 0;
  EntryId entry_id = 0;
};

static_assert(sizeof(TerPartitionEntry) == 8);
static_assert(std::is_trivially_copyable_v<TerPartitionEntry>);

enum class PartitionBackend : std::uint8_t {
  CurrentScatter,
  BlockInPlace,
  Region2Path,
  Adaptive,
```

```
};

struct PartitionSpec {
  std::uint8_t shift = 0;
  std::uint8_t bits = 8;
  std::uint32_t threads = 1;
  PartitionBackend backend = PartitionBackend::CurrentScatter;
  std::uint64_t scratch_cap_bytes = 0;
  std::uint32_t block_bytes = 2048;
};

struct PartitionStats {
  PartitionBackend requested = PartitionBackend::CurrentScatter;
  PartitionBackend effective = PartitionBackend::CurrentScatter;

  std::uint64_t histogram_ns = 0;
  std::uint64_t prefix_ns = 0;
  std::uint64_t scatter_ns = 0;
  std::uint64_t local_classification_ns = 0;
  std::uint64_t block_permute_ns = 0;
  std::uint64_t cleanup_ns = 0;

  std::uint64_t records_classified = 0;
  std::uint64_t records_written = 0;
  std::uint64_t blocks_flushed = 0;
  std::uint64_t blocks_permuted = 0;
  std::uint64_t partial_records_repaired = 0;

  std::uint64_t histogram_bytes = 0;
  std::uint64_t offset_bytes = 0;
  std::uint64_t record_scratch_bytes = 0;
  std::uint64_t block_buffer_bytes = 0;
  std::uint64_t scratch_peak_bytes = 0;

  std::uint64_t logical_bytes_read = 0;
  std::uint64_t logical_bytes_written = 0;
};

class PartitionWorkspace {
 public:
  PartitionWorkspace();
  PartitionWorkspace(const PartitionWorkspace&) = delete;
  PartitionWorkspace& operator=(const PartitionWorkspace&) = delete;
  PartitionWorkspace(PartitionWorkspace&&) noexcept;
  PartitionWorkspace& operator=(PartitionWorkspace&&) noexcept;
  ~PartitionWorkspace();

 private:
  struct Impl;
  std::unique_ptr<Impl> impl_;
  friend Status partition_chunk(
      std::span<TerPartitionEntry>, const PartitionSpec&, PartitionWorkspace&,
      std::span<std::uint64_t>, PartitionStats&);
};

[[nodiscard]] Status partition_chunk(
    std::span<TerPartitionEntry> entries,
    const PartitionSpec& spec,
```

```
    PartitionWorkspace& workspace,
    std::span<std::uint64_t> bucket_offsets,
    PartitionStats& stats);

}  // namespace ter
```

Implementation note: include `<memory>` in the real header because `std::unique_ptr` is used.

The opaque workspace prevents the public header from leaking backend internals and permits capacity reuse across A/B/C partitions.

8 Input validation and statuses

`partition_chunk` MUST validate before mutating entries:

1. `bits` is 8, 12, or 16 for CurrentScatter;
2. `bits == 8` for BlockInPlace v1;
3. `shift + bits <= 32`;
4. `threads >= 1`;
5. `bucket_offsets.size() == (1ULL << bits) + 1`;
6. `block_bytes` is a power of two and at least `sizeof(TerPartitionEntry)` for BlockInPlace;
7. `block_bytes % sizeof(TerPartitionEntry) == 0`;
8. required scratch fits `scratch_cap_bytes` when the cap is nonzero;
9. all count/byte arithmetic uses overflow-checked helpers;
10. cancellation, if later added to the spec, is checked at safe phase boundaries.

Required status behavior:

- success: `Status::Completed`;
- candidate backend deliberately unsupported: `Status::NotApplicable`;
- physical representation cannot support requested width: `Status::UnsupportedWidth`;
- memory admission failure: `Status::ResourceLimit`;
- invariant failure after mutation starts: `Status::InvariantViolation`;
- invalid caller configuration: `Status::InvalidConfiguration`.

This requires append-only `NotApplicable` and `UnsupportedWidth` additions to the global `Status` enum before P2 is exposed.

9 CurrentScatter backend

CurrentScatter is the exact baseline. It is intentionally boring.

9.1 Serial path

For $B = 2^b$:

```
1. resize/zero global counts[B]
2. scan records and count digit(record)
3. exclusive prefix -> bucket_offsets[B+1]
4. copy bucket_offsets[0..B-1] to write cursors
5. resize full-N record scratch
6. scan records and write complete record to scratch[cursor[digit]++]
7. copy/swap scratch back into entries
8. validate debug invariants if enabled
```

9.2 Parallel path

Adapt the donor radix worker model, not an unrelated parallel framework.

For each worker:

```
source range = deterministic contiguous stripe
private histogram[B]
private write offsets[B]
```

Global prefix is bucket-major, then worker-major, yielding disjoint output subranges for every worker/bucket pair. Parallel scatter then writes complete `TerPartitionEntry` records without destination contention.

Required accounting:

- `records_classified == N`;
- `records_written == N`;
- `record_scratch_bytes >= N*sizeof(TerPartitionEntry)` when N>0;
- `histogram_bytes` and `offset_bytes` report **capacity-backed allocated bytes**, not merely logical element count, if that is the existing TER accounting convention;
- timings use `steady_clock` nanoseconds;
- stats are reset at function entry.

9.3 CurrentScatter physical widths

Support exactly:

```
8, 12, 16 bits
```

Do not implement a dense 24-bit parallel histogram.

10 BlockInPlace backend - source boundary

BlockInPlace is not "call ips2ra::sort". It is a TER-native one-level partition backend derived from the blockwise distribution principles and selected source mechanics of IPS2Ra.

10.1 IPS2Ra source pieces that are in scope

Read and adapt concepts from:

- `partitioning.hpp`: phase order `classification -> block permutation -> cleanup`;
- `local_classification.hpp`: per-bucket buffering and bucket-size accumulation;
- `block_permutation.hpp`: classify displaced blocks and move them through target bucket ranges;
- `cleanup_margins.hpp`: repair partial bucket boundaries/buffers after full-block permutation;
- `bucket_pointers.hpp`: synchronized read/write block ownership semantics;
- `config.hpp`: starting `kLogBuckets=8`, `kBlockSizeInBytes=2048`, minimum parallel blocks per thread idea;
- `memory.hpp` / `buffers.hpp`: aligned per-thread bucket buffers and swap blocks.

10.2 IPS2Ra pieces explicitly out of scope

Do not import:

- recursive full sorting;
- sample-level discovery;
- dynamic radix-level recursion;
- TBB scheduler or task queues;
- base-case comparison sorting;
- IPS4o sampling machinery;
- whole-repository CMake targets;
- OpenMP as a required dependency;
- direct external-link dependency on the local clone;
- the assumption that Windows can build upstream IPS2Ra unchanged.

11 BlockInPlace v1 physical design

BlockInPlace v1 is **radix8 only**:

```
bits = 8
B = 256 buckets
block_bytes default = 2048
record_bytes = 8
records_per_block = 256
```

The 2048 byte starting block size is inherited from the upstream IPS2Ra default and is a starting configuration, not a TER optimum. The benchmark MUST later sweep at least 1024, 2048, and 4096 bytes.

At 16 workers with one full 2048-byte buffer per bucket/worker, the raw buffer ceiling is:

$$16 \cdot 256 \cdot 2048 = 8 \text{ MiB}.$$

This is far below a 5M-record CurrentScatter scratch array of 38.15 MiB, while still requiring measurement because extra record movement may cost time.

12 BlockInPlace algorithm - exact phase contract

12.1 Phase BI-1: local classification and buffered flushing

Each worker owns a deterministic contiguous stripe. It scans each record exactly once for classification.

Per worker:

- `bucket_size[256]` counts all records by digit;
- `buffer[256]` stores up to one block of records per digit;
- full buffers are flushed into aligned full-block slots in the worker's local classified prefix;
- partial buffers remain in worker-local memory until global cleanup;
- `entry_id` moves with `low_k` in every copy/move.

The classification function is fixed and branch-simple:

```
inline std::uint32_t digit_of(const TerPartitionEntry& e,
                              std::uint8_t shift) noexcept {
  return (e.low_k >> shift) & 0xFFu;
}
```

No sampling or comparison is permitted.

12.2 Phase BI-2: global bucket boundaries

Reduce per-worker bucket counts into exact global counts and compute exclusive prefix:

```
offsets[0] = 0
offsets[j+1] = offsets[j] + global_count[j]
offsets[256] = N
```

All additions are overflow-checked before mutation continues.

12.3 Phase BI-3: bucket block ownership

For each bucket, maintain logical write/read block positions covering only full aligned blocks inside that bucket's final country.

The portable TER implementation MAY copy the upstream `BucketPointers` algorithmic semantics, but it MUST NOT require `unsigned __int128` or 16-byte atomic CAS on Windows.

Portable rule:

- serial path uses plain integer pointers;
- parallel path uses a small mutex-protected pair of read/write indices plus an atomic reader counter unless a later benchmarked platform specialization is explicitly enabled;
- correctness comes before lock-free cleverness;

- any future 128-bit atomic specialization must be behind a feature macro and differential benchmark.

12.4 Phase BI-4: full-block permutation

For each source bucket with unread full blocks:

1. claim/read one full block;
2. classify the block head to obtain destination bucket;
3. claim one full write position in destination bucket;
4. if destination slot is empty, write and finish chain;
5. if destination slot contains a block belonging elsewhere, swap it into the worker swap buffer;
6. continue the displaced-block chain until it lands in an empty/correct position;
7. never overwrite a block another worker is currently reading.

Required progress invariant:

> every completed displacement step places at least one full block into a destination range consistent with its digit, or moves the displaced block closer to the same invariant without duplicating/dropping data.

The backend MUST bound/validate progress in debug/test builds and return `InvariantViolation` rather than spinning forever.

12.5 Phase BI-5: partial-boundary cleanup

After full blocks are positioned:

- repair bucket heads/tails that are not aligned to full blocks;
- drain every worker's partial per-bucket buffers into the exact remaining slots;
- repair any overflow/swap-buffer fragment;
- reset reusable worker buffers without deallocating capacity;
- assert exact final bucket membership in debug/property tests.

No recursive sorting follows cleanup.

13 Worker pool and portability

The preferred threading substrate is the existing donor-style `RadixWorkerPool` based on `std::thread`, condition variables, deterministic generation IDs, and one synchronous task per pass.

The TER CMake change SHOULD use:

```
find_package(Threads REQUIRED)
target_link_libraries(ter_t0 PUBLIC Threads::Threads)
```

Do not add TBB. Do not add the local `ips2ra` directory as a submodule. Do not require OpenMP.

14 PartitionWorkspace internals

`PartitionWorkspace::Impl` SHOULD own reusable capacity for both backends:

```
CurrentScatter:
  record_scratch
  global_counts
  global_cursors
  worker histogram vectors
  worker offset vectors
  persistent worker pool

BlockInPlace:
  global_counts
  worker bucket counts
  aligned per-worker bucket buffers
```

```
two swap blocks per active worker
one overflow block per active worker or shared phase object
per-bucket read/write ownership state
per-worker local bookkeeping
```

Workspace reuse MUST avoid deallocation between A/B/C partitions unless a memory cap forces release.

The workspace MUST expose no algorithm-specific containers in the public header.

15 Memory admission

Before any backend mutates `entries`, it MUST estimate peak required auxiliary bytes with checked arithmetic.

15.1 CurrentScatter estimate

Include at minimum:

```
N * sizeof(TerPartitionEntry)
+ global bucket arrays
+ per-worker histogram capacity
+ per-worker offset capacity
+ bounded worker-pool state
```

15.2 BlockInPlace estimate

Include at minimum:

```
threads * 256 * block_bytes
+ swap/overflow blocks
+ global counts/offsets
+ worker bucket counts
+ bucket ownership state
+ alignment overhead
```

If `scratch_cap_bytes != 0` and estimated peak exceeds the cap, return `ResourceLimit` before mutation.

16 Telemetry - common and backend-specific

Common fields:

```
backend requested/effective
physical shift/bits
threads requested/used
N
histogram_ns
prefix_ns
records_classified
histogram_bytes
offset_bytes
scratch_peak_bytes
logical_bytes_read
logical_bytes_written
```

CurrentScatter adds:

```
scatter_ns
records_written
record_scratch_bytes
```

BlockInPlace adds:

`local_classification_ns`
`block_permute_ns`
`cleanup_ns`
`blocks_flushed`
`blocks_permuted`
`partial_records_repaired`
`block_buffer_bytes`

Telemetry MUST distinguish logical bytes moved from allocator capacity. It MUST not claim DRAM traffic without hardware counters.

17 Correctness protocol

Every timed backend first passes untimed property tests.

For an input with unique IDs:

$$multiset(output\ IDs) = multiset(input\ IDs).$$

For every bucket j:

$$\forall e \in E[offsets[j], offsets[j+1]) : digit(e) = j.$$

Required fixtures:

- N=0;
- N=1;
- all records in one bucket;
- one record per bucket;
- every bucket populated;
- 99.99% hot bucket;
- alternating two buckets;
- already partitioned;
- reverse partitioned;
- random EntryIds unrelated to key order;
- `EntryId` maximum value fixtures;
- duplicate `low_k` with distinct IDs;
- random N across block boundaries: 255, 256, 257, 511, 512, 513;
- stripe/thread boundaries around `threads * records_per_block`;
- memory-cap exact-fit and one-byte-over tests.

17.1 Differential oracle

CurrentScatter is the mandatory oracle for BlockInPlace. The comparison is **bucket membership/counts + EntryId multiset**, not within-bucket order.

For every test input:

1. clone input;
2. run CurrentScatter;
3. run BlockInPlace;
4. compare bucket offsets exactly;
5. sort EntryIds within corresponding bucket spans only in test code and compare equality;
6. verify no input mutation occurred on preflight `ResourceLimit`/invalid-spec paths.

18 Benchmark protocol

18.1 Dataset matrix

Required N:

```
20k, 32,496, 64k, 0.5M, 1M, 2M, 5M
```

Required distributions:

```
uniform
Zipf-like
99.99% hot bucket
duplicate-heavy
already partitioned
reverse partitioned
adversarial alternating buckets
captured genuine L1 A/B/C lists
```

Required threads:

```
1, 2, 4, 8, 16
```

Initial physical comparisons:

```
8-bit CurrentScatter vs 8-bit BlockInPlace
16-bit CurrentScatter as a separate baseline
n=72 low24 schedule: 16 CurrentScatter -> prune -> 8 BlockInPlace survivors
```

Experimental later schedules:

```
8 + 8 + 8
12 + 12 only after a 12-bit block-buffer design is explicitly implemented
```

18.2 Paired timing

For each repetition, use the same input instance for both backends and randomize backend order. Run 5 warmups + at least 30 measured paired repetitions.

For paired speedup $S_i = T_{current,i}/T_{candidate,i}$, aggregate in log space:

$$\hat{S} = \exp\left(\frac{1}{m}\sum_i \ln S_i\right).$$

Bootstrap paired log ratios 10,000 times for a 95% confidence interval.

18.3 Promotion gates

Speed default: candidate becomes default only if lower 95% CI of paired geometric-mean speedup is > 1.05x on representative genuine L1 captures and no serious tail regression appears.

Memory backend: candidate may be retained as a memory backend if peak partition scratch is reduced by at least 4x while wall time stays within 10% of CurrentScatter.

If neither gate is met, CurrentScatter remains production baseline and BlockInPlace remains experimental/reference code.

19 Adaptive backend - deferred contract

`PartitionBackend::Adaptive` exists in the enum to avoid API churn but returns `Status::NotApplicable` until the benchmark phase installs thresholds.

When enabled later, its decision inputs are limited to observable values:

```
N
bits
threads
scratch cap
predicted CurrentScatter scratch
bucket skew/nonempty count/max occupancy
measured backend model coefficients
```

No ML router is permitted at this stage.

20 Region2Path - deferred comparator contract

`Region2Path` remains scientifically useful because it tests the graph-based Regions Sort mechanism against block permutation. It is not part of the first coding milestone.

If later implemented:

- physical radix8 first;
- only one-level local distribution + country/region graph + 2-path relocation;
- no recursion;
- no Cilk scheduler;
- no 16-bit graph with 65,536 countries as default;
- same `partition_chunk` postcondition and telemetry;
- same CurrentScatter differential oracle.

21 Licensing and source-provenance rules

The TER repository is MIT licensed. IPS2Ra source files are BSD 2-Clause. Copy/adapt is permitted only while preserving the required copyright/license notice.

Implementation requirements:

1. Add `third_party/licenses/ips2ra-BSD-2-Clause.txt` containing the upstream BSD-2-Clause license text.
2. Every TER source file containing substantial copied/adapted IPS2Ra code MUST retain the upstream copyright/license notice or a clear SPDX/provenance header plus the full notice in the third-party license file, consistent with BSD-2-Clause requirements.
3. Comments MUST distinguish "adapted from IPS2Ra" from independently written TER code.
4. Do not remove TER's root MIT license.
5. Record upstream repository and reference commit in the derived source header.

22 File layout - frozen recommendation

Create:

```
include/ter/root_partition.hpp
src/root_partition.cpp
src/root_partition_current_scatter.cpp
src/root_partition_block_inplace.cpp
tests/root_partition_tests.cpp
bench/ter_root_partition_bench.cpp
bench/analyse_root_partition.py
third_party/licenses/ips2ra-BSD-2-Clause.txt
```

Responsibilities:

- `root_partition.hpp`: public API only;
- `root_partition.cpp`: validation, dispatch, workspace pImpl, common telemetry helpers;
- `root_partition_current_scatter.cpp`: donor-derived histogram/prefix/scatter baseline;
- `root_partition_block_inplace.cpp`: IPS2Ra-derived radix8 block classification/permutation/cleanup;
- `root_partition_tests.cpp`: property/differential tests;

- `ter_root_partition_bench.cpp`: isolated deterministic microbenchmark and captured-root replay adapter;
- `analyse_root_partition.py`: paired CI/statistics only, no benchmark generation;
- third-party license file: source obligations.

Do not create a monolithic `sorter.cpp` containing both complete source repositories.

23 Build integration

Modify `CMakeLists.txt` to:

- add the three partition `.cpp` files to `ter_t0`;
- `find_package(Threads REQUIRED)`;
- link `Threads::Threads`;
- add `ter_root_partition_tests` and register it with CTest;
- add `ter_root_partition_bench` without registering it as a test;
- preserve existing strict warnings and IPO/native options;
- add no TBB/IPS2Ra subdirectory.

24 Implementation order and stop gates

24.1 H0 - safety audit

No code changes until:

- local TER `git status --short` recorded;
- unique local `ssp_oracle.cpp` located;
- IPS2Ra clone checked read-only;
- current full tests pass or pre-existing failures are recorded.

24.2 H1 - API + status + tests

Introduce append-only statuses, public API, validation tests. No backend algorithm yet except empty/singleton trivial path.

24.3 H2 - CurrentScatter serial

Implement exact one-chunk serial baseline and property tests.

24.4 H3 - CurrentScatter parallel

Port donor worker state/pool/prefix/scatter and verify deterministic bucket membership.

24.5 H4 - isolated benchmark

Benchmark CurrentScatter before BlockInPlace so the baseline is not rewritten after seeing candidate results.

24.6 H5 - BlockInPlace serial reference

Implement radix8 local classification/block permutation/cleanup without concurrency first. Differential tests must pass.

24.7 H6 - BlockInPlace parallel

Add worker ownership using the portable pointer synchronization contract. Differential + stress + sanitizer gates.

24.8 H7 - paired benchmark and decision

Only now compare speed/memory. No Adaptive thresholds before this point.

24.9 H8 - root Stage C integration

Use `partition_chunk` to build contiguous EntryId spans/CSR directories for root work. Do not implement Stage D pruning in the same commit.

25 Absolute no-go list

Codex MUST NOT:

- reset/clean the worktree and erase uncommitted `ssp_oracle.cpp`;
- import the whole IPS2Ra repository into TER;
- add IPS2Ra as a submodule;
- modify `C:\Users\naman\Desktop\ips2ra`;
- add TBB solely for this feature;
- require upstream IPS2Ra to compile on Windows;
- sort full TER entries instead of 8-byte movement records;
- sort by `(low_k, EntryId)` as a combined numeric key;
- run extra radix passes to order EntryIds;
- assume stable within-bucket order;
- allocate a 24-bit dense per-worker histogram;
- implement Region2Path before the BlockInPlace bake-off;
- hard-code Adaptive thresholds without data;
- claim the modelled 1.15x/1.25x/1.50x sensitivities as measured;
- publish IPS2Ra-derived code without its BSD notice;
- conflate partition speedup with root pair-space reduction.

26 Final P2 acceptance checklist

☐ `TerPartitionEntry` is exactly 8 bytes and trivially copyable.
☐ CurrentScatter uses `low_k` for classification and moves whole records.
☐ Every successful partition returns exact monotone offsets covering N.
☐ EntryId multiset is preserved exactly.
☐ CurrentScatter supports 8/12/16 physical bits.
☐ BlockInPlace v1 supports exactly 8 bits.
☐ BlockInPlace is TER-native and has no TBB/submodule dependency.
☐ IPS2Ra-derived portions preserve BSD-2-Clause provenance.
☐ All property/differential tests pass before benchmark.
☐ CurrentScatter baseline timing was frozen before candidate tuning.
☐ Paired benchmark uses same inputs and randomized order.
☐ Speed/memory decisions use the stated gates.
☐ Region2Path remains deferred unless the specs are revised.
☐ Adaptive remains `NotApplicable` until measurements justify thresholds.
☐ Root integration consumes bucket spans/EntryIds without moving cold TER entries.